Password Analyzer Project Report

Princess Nourah bint Abdulrahman University

College of Computer and Information Sciences

Project Members

Student Name	***Percentage***
Razan Alshahrani	25%
Team member 1	25%
Team member 2	25%
Team member 3	25%

1. Introduction

This project is a Password Analyzer developed using Java. It ensures that user-provided passwords meet strict security standards. If a password fails validation, the system generates a strong password that approves to all criteria.

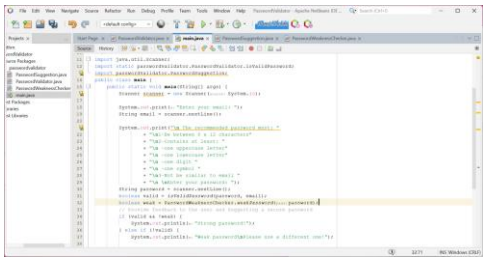
2. Password Requirements

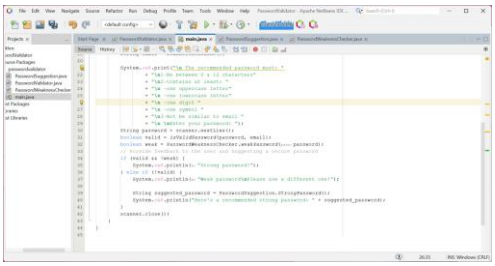
- Password length must be between 8 to 12 characters
- Must contain: one uppercase letter, one lowercase letter, one digit, one symbol
- Must not be similar to email
- No dictionary words
- No repetitive or sequential characters

3. Design and Implementation

The system uses these Java components:

1. ***main Class (Main Program)***





Purpose:

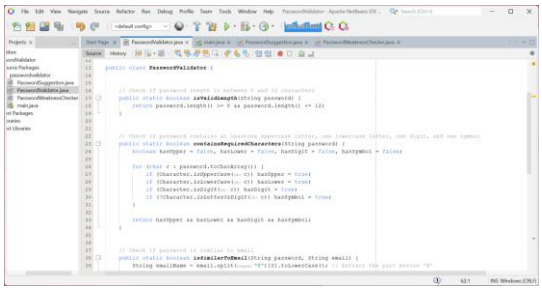
Acts as the starting point of the program and manages the flow of the password validation process.

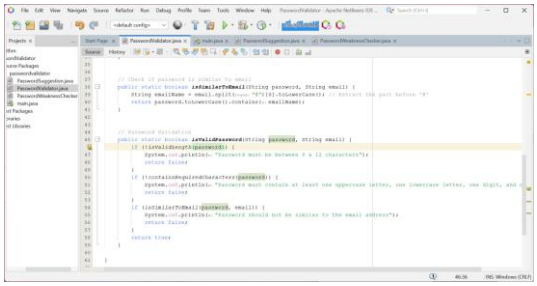
What it does:

- Prompts the user to enter their email and password.
- Displays the password rules and requirements clearly.
- Calls isValidPassword() from PasswordValidator to check if the password follows the structure rules.
- Calls weakPassword() from PasswordWeaknessChecker to identify any weak patterns.

- If the password is valid and strong, it prints a success message.
- If the password is invalid or weak:
- It prints why the password failed.
- It calls StrongPassword() from PasswordSuggestion to suggest a secure alternative

2.PasswordValidator Class





Purpose:

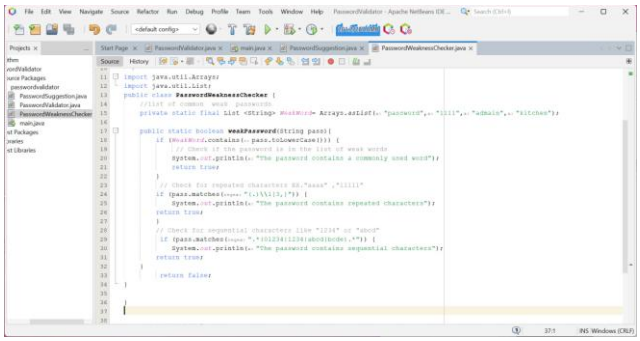
Validates the format and structure of the password to ensure it meets security criteria.

What it checks:

- isValidLength(password)
- Verifies that the password length is between 8 and 12 characters.
- containsRequiredCharacters(password)
- Confirms that the password contains:
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special symbol
- isSimilarToEmail(password, email)
- Ensures the password does not contain the email's username (the part before @).
- isValidPassword(password, email)

- Combines all above methods.
- If any rule fails, it prints the reason and returns false

3.**PasswordWeaknessChecker Class**



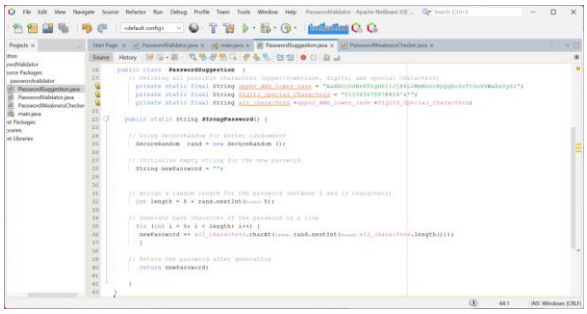
Purpose:

Checks if the password contains weak or predictable patterns.

What it detects:

- Common weak words like "password", "admin", "1111", or "kitchen".
- Repetitive characters, such as "aaaa" or "1111", using regex.
- Sequential characters, like "1234" or "abcd", which are easy to guess.
- weakPassword(String pass)
- Runs all checks.
- If any weakness is found, it prints the reason and returns true

4. **PasswordSuggestion Class**



Purpose:

Generates a strong password when the user input is weak or invalid.

How it works:

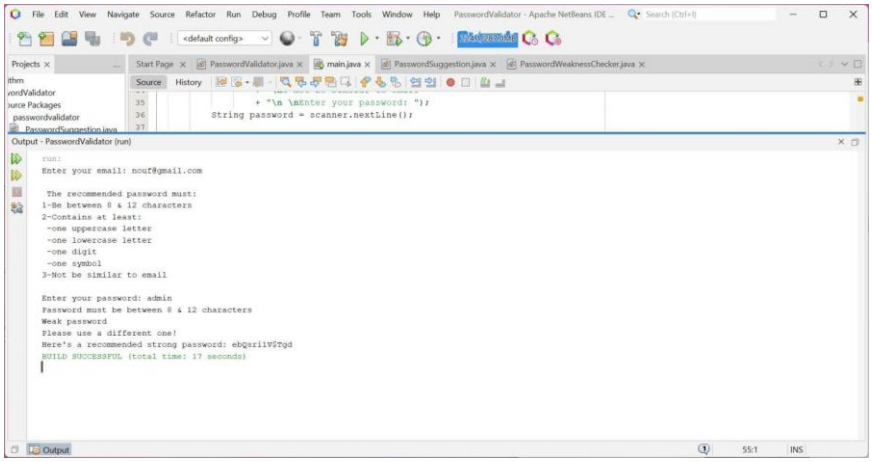
- Uses SecureRandom to generate truly random characters.
- Selects a password length between 8 and 12 characters randomly.
- Mixes:
- Uppercase and lowercase letters
- Numbers
- Special characters (e.g., !@#$%^&*)
- Ensures that the password includes all required types of characters.
- StrongPassword()
- Returns a password that meets all conditions and avoids weak patterns

4. Sample Outputs

Case 1

Email Entered	nouf@gmail.com
Password Entered	admin
Issue Detected	Password is too short and lacks all required character types (uppercase, digit, symbol). A strong password is suggested.

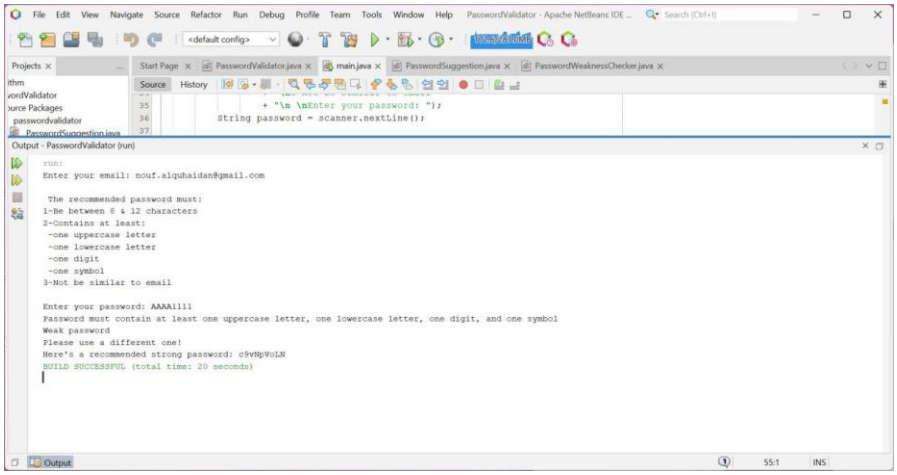
Output Screenshot



Case 2

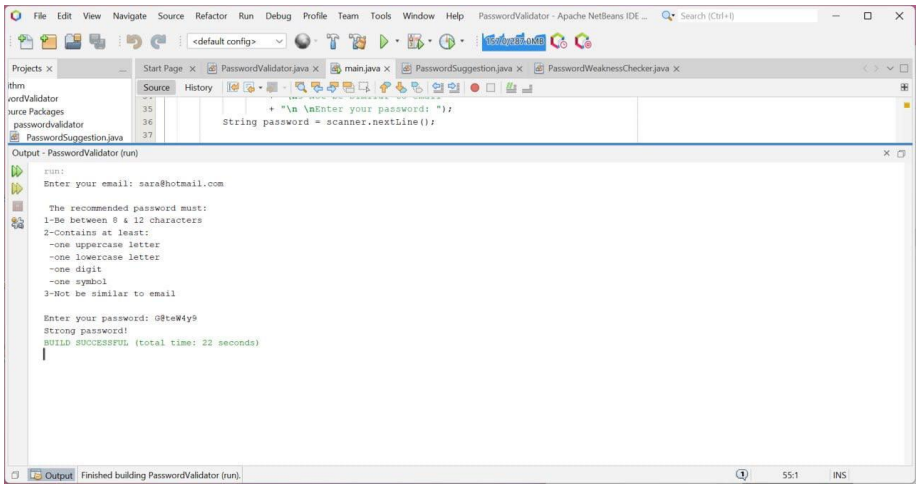
Email Entered	nouf.alquhaidan@gmail.com
Password Entered	AAAA1111
Issue Detected	Password lacks lowercase letters and symbols, making it weak. System identifies weakness and suggests a strong password and has repetitive characters.

Output Screenshot



Case 3

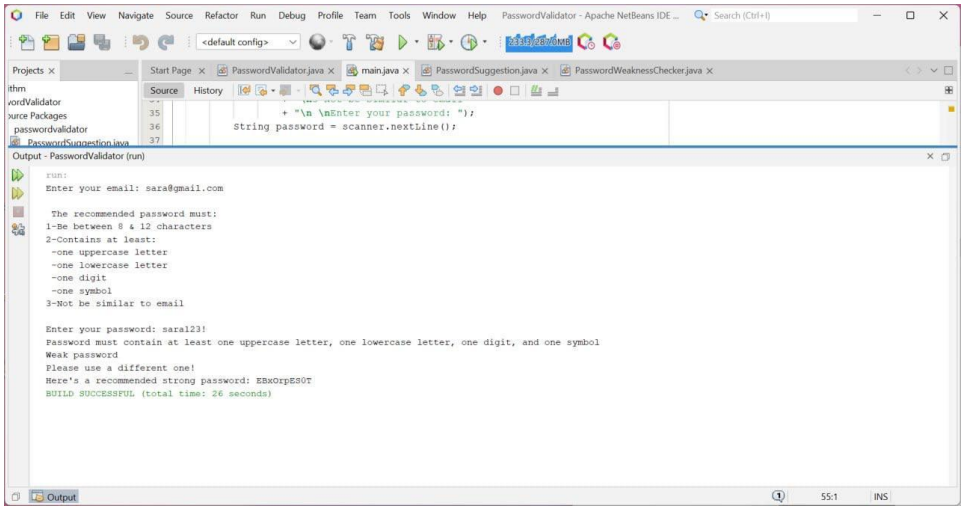
Email Entered	sara@hotmail.com
Password Entered	G@tew4y9

Result:	Meets all criteria: Length: 8 characters,Contains uppercase, lowercase, digit, and symbol Not similar to email.
Output Screenshot	

Case 4

Email Entered	sara@gmail.com
Password Entered	sara123!
Result:	Password is too similar to email Verdict: Weak password,Suggested Password: EBxOrpES0T

Output Screenshot



5. Challenges and Solutions

Throughout the project, one of the trickiest parts was making sure the password checks worked exactly as they should—especially when trying to catch things like weak patterns or passwords that were too similar to the user's email. It took some trial and error to get everything right. Breaking the code into smaller, focused parts really helped us stay organized and made the tough parts easier to handle. Working together and talking through our ideas also made a big difference when things got confusing.

6. Conclusion

Working on the Password Analyzer project gave us a chance to tackle a real-world security issue using what we've learned in Java. We built a system that doesn't just check if a password follows certain rules—it also looks for common weaknesses and helps users choose stronger options. It was really rewarding to see how something as simple as checking a password can turn into a full-featured program with real impact.

Throughout the process, we learned a lot—not just about coding with Java, but also about how to think critically about user input, design clean and organized code, and create something that's both secure and user-friendly. Breaking things down into clear modules like validation, weakness checking, and password suggestions helped us stay organized and made the whole project easier to manage.

This project taught us how powerful programming can be when it's used to solve everyday problems. And more than that, it reminded us of the importance of good teamwork and communication.